Far-Field Transform via Vector Cylindrical Harmonics: Reformulation for Large Oblique Angles

FFTLog-Based Hankel Transforms and Local Focal-Plane Basis

Julia Implementation

Based on notes by Arvin Keshvari (Supervisor: Dr. Zin Lin)

March 26, 2026

Contents

1 Why the Original Formulation Struggles at Large Angles **2**
1.1 Mode count grows with oblique angle . . . 2
1.2 Two computational bottlenecks . . . 2

2 Improvement 1 — FFTLog for the Forward Hankel Transforms **2**
2.1 The FFTLog idea . . . 2
2.2 Reformulated forward transforms . . . 3

3 TM Coefficient (completing the original notes) **3**

4 Improvement 2 — Local Basis Centered at the Focal Spot **4**
4.1 Angular spectrum representation . . . 4
4.2 Focal-spot shift . . . 4
4.3 Polar-to-Cartesian regridding . . . 4

5 Complete Improved Algorithm **4**

6 Julia Implementation Notes **5**
6.1 Package dependencies . . . 5
6.2 FFTLog Hankel transform . . . 5
6.3 Angular decomposition (Step 1) . . . 6
6.4 Forward Hankel transforms for TE and TM coefficients (Step 2) . . . 7
6.5 Angular spectrum assembly and propagation (Steps 3–4) . . . 8
6.6 Polar-to-Cartesian regridding (Step 5) . . . 9
6.7 Local PSF (Step 6) . . . 10
6.8 Top-level driver . . . 11

7 Comparison Table **12**
7.1 Symmetry reduction 12
7.2 Large-angle validity 12

Why the Original Formulation Struggles at Large Angles

Mode count grows with oblique angle

For a metalens of radius R illuminated at oblique angle α, the near field contains the phase factor $e^{ik\sin\alpha\, r\cos\theta}$. The angular momentum decomposition

$$u_m(r) = \frac{1}{2\pi}\int_0^{2\pi} u(r,\theta)\, e^{-im\theta}\, d\theta$$

picks out contributions from the Jacobi–Anger expansion. For the ideal oblique lens (scalar case),

$$u_m(r) = 2\pi(-i)^m \exp\!\big(-ik[\sqrt{r^2+f^2}-f]\big)\, J_m(k\sin\alpha\, r).$$

Since $J_m(x) \approx 0$ for $m \gg x$, the required mode truncation is

$$\boxed{M_{\max} \approx k\sin\alpha\cdot R = \frac{2\pi\sin\alpha\cdot R}{\lambda}.}$$

Numerical example. For $R = 500\lambda$, $\alpha = 11$ ($\sin\alpha \approx 0.19$): $M_{\max} \approx 600$. For $\alpha = 30$: $M_{\max} \approx 1570$. These are the Bessel orders ν that must be handled in the Hankel transforms.

Two computational bottlenecks

1. **High-order Hankel transforms.** The forward transforms $a_m(k_r) = k_r \int_0^\infty J_m(k_r r)\, u_m(r)\, r\, dr$ reach order $|m| = M_{\max} \sim 600$. Algorithms built on large-argument or small-argument Bessel asymptotics (e.g. Beckman–O'Neil NUFHT) break down near the turning point $k_r r \approx |m|$, which at $|m| = 600$ spans the bulk of the integration range.

2. **Off-axis PSF evaluation.** The focal spot sits at $(x_0, 0) = (f\tan\alpha, 0)$. Computing the cylindrical-harmonic sum $u(r,\theta,f) = \sum_m [\ldots] e^{im\theta}$ near that point requires evaluating $M_{\max}$ terms at large argument, which is costly and numerically fragile.

Improvement 1 — FFTLog for the Forward Hankel Transforms

The FFTLog idea

FFTLog (Talman 1978; Hamilton 2000) computes

$$A(k_r) = \int_0^\infty J_\nu(k_r r)\, f(r)\, r\, dr$$

by changing variables to $r = r_0 e^{-\rho}$, $k_r = k_0 e^{\kappa}$, turning the transform into a convolution:

$$\tilde{A}(\kappa) = \int_{-\infty}^{\infty} \tilde{f}(\rho)\, \tilde{J}_\nu(\kappa-\rho)\, d\rho,$$

evaluated by FFT. The filter kernel in Fourier space is

$$\hat{\tilde{J}}_\nu(q) = \frac{\Gamma\left(\frac{\nu+1+q}{2}\right)}{\Gamma\left(\frac{\nu+1-q}{2}\right)}\, 2^q,$$

which is analytic for *any* real ν, including $\nu = 600$. **There is no turning-point problem: FFTLog is order-agnostic.**

Julia implementation. The function `fftlog_hankel(f_r, r, nu)` in `CyFFP.jl` implements FFTLog using `FFTW.fft/ifft` and `SpecialFunctions.loggamma` for the Gamma-function ratio. For $|\nu| \sim 600$ the ratio $\Gamma((\nu+1+q)/2)/\Gamma((\nu+1-q)/2)$ is computed in log-space to avoid overflow, using the identity $\log|\hat{\tilde{J}}_\nu(q)| = \mathrm{Re}[\mathrm{loggamma}((\nu+1+q)/2) - \mathrm{loggamma}((\nu+1-q)/2)] + q\log 2$. Input `f_r` must be sampled on a logarithmic grid `r = r0 .* exp.(dln .* (0:N-1))`.

Reformulated forward transforms

The scalar coefficient is

$$a_m(k_r) = k_r \cdot \mathcal{H}_m\big[r\, u_m(r)\big](k_r),$$

evaluated by a single FFTLog call of order m, costing $O(N \log N)$ regardless of m.

For the vector TE coefficient the recurrence relations $J_{m\pm1}(x) = (m/x)J_m(x) \mp J'_m(x)$ and the dot-product expansion from the original notes give:

$$A_m^{\mathrm{TE}}(k_r) = \frac{i}{2}\big[\mathcal{H}_{m+1}[r\, E^{\mathrm{N}}_{m,r}] + \mathcal{H}_{m-1}[r\, E^{\mathrm{N}}_{m,r}]\big] + \frac{1}{2}\big[\mathcal{H}_{m-1}[r\, E^{\mathrm{N}}_{m,\theta}] - \mathcal{H}_{m+1}[r\, E^{\mathrm{N}}_{m,\theta}]\big]. \tag{1}$$

The TM coefficient follows analogously (see Section 3). Each mode requires **4 FFTLog calls** (6 including the z-component).

TM Coefficient (completing the original notes)

The TM electric harmonic is (from the original notes, Eq. 34):

$$\boldsymbol{E}^{\mathrm{TM}}_{m,k_r} = \sqrt{\tfrac{\mu}{\epsilon}}\left[-\tfrac{k_z}{k}\tfrac{\partial J_m(k_r r)}{\partial r}\,\hat{\boldsymbol{r}} - i\tfrac{mk_z}{k_r r}\,J_m(k_r r)\,\hat{\boldsymbol{\theta}} + i\tfrac{k_r^2}{k}\,J_m(k_r r)\right] e^{im\theta}e^{-ik_z z}.$$

The tangential part (projecting onto $\hat{\boldsymbol{r}}, \hat{\boldsymbol{\theta}}$) is:

$$\boldsymbol{E}^{\mathrm{TM},\perp}_{m,k_r} = -\tfrac{k_z}{k}\tfrac{\partial J_m}{\partial r}\,\hat{\boldsymbol{r}} - i\tfrac{mk_z}{k_r r}\,J_m\,\hat{\boldsymbol{\theta}}.$$

Using the same recurrence relations as for the TE case:

$$\tfrac{\partial J_m}{\partial r} = \tfrac{k_r}{2}(J_{m-1} - J_{m+1}), \quad \tfrac{m}{k_r r}J_m = \tfrac{1}{2}(J_{m-1} + J_{m+1}),$$

the projection integral becomes:

$$A_m^{\mathrm{TM}}(k_r) = -\frac{k_z/k}{2}\big[\mathcal{H}_{m-1}[r\, E^{\mathrm{N}}_{m,r}] - \mathcal{H}_{m+1}[r\, E^{\mathrm{N}}_{m,r}]\big] - \frac{ik_z/k}{2}\big[\mathcal{H}_{m-1}[r\, E^{\mathrm{N}}_{m,\theta}] + \mathcal{H}_{m+1}[r\, E^{\mathrm{N}}_{m,\theta}]\big]. \tag{2}$$

Note that $k_z = k_z(k_r) = \sqrt{k^2 - k_r^2}$ so the TM coefficient depends on k_z/k, which must be evaluated at each k_r grid point.

Improvement 2 — Local Basis Centered at the Focal Spot

Angular spectrum representation

Using the Jacobi–Anger identity one shows that the far field is the 2D inverse Fourier transform of the angular spectrum:

$$u(x, y, f) = \frac{1}{(2\pi)^2} \iint \mathcal{U}(k_x, k_y)\, e^{i(k_x x + k_y y)}\, dk_x\, dk_y,$$

where the propagation phase $e^{ik_z f}$ has already been absorbed into $\mathcal{U}$, and

$$\mathcal{U}(k_r, \phi) = \frac{2\pi}{k_r} \sum_m (-i)^m a_m(k_r)\, e^{im\phi}.$$

This is an **inverse FFT over** m at each k_r ring — cheap and exact.

Focal-spot shift

The focal spot for oblique angle α sits at $\boldsymbol{x}_0 = (f \tan\alpha,\, 0)$. Define local focal-plane coordinates:

$$\xi = x - x_0, \quad \eta = y,$$

and the *shift-centered angular spectrum*:

$$\mathcal{G}(\tilde{k}_x, \tilde{k}_y) = \mathcal{U}(k \sin\alpha + \tilde{k}_x,\, \tilde{k}_y)\, e^{i(k \sin\alpha + \tilde{k}_x)\, x_0}\, e^{ik_z f},$$

where $k_z = \sqrt{k^2 - (k \sin\alpha + \tilde{k}_x)^2 - \tilde{k}_y^2}$. The local PSF is then:

$$\boxed{\mathrm{PSF}(\xi, \eta) = \frac{e^{ik \sin\alpha \cdot \xi}}{(2\pi)^2} \iint \mathcal{G}(\tilde{k}_x, \tilde{k}_y)\, e^{i(\tilde{k}_x \xi + \tilde{k}_y \eta)}\, d\tilde{k}_x\, d\tilde{k}_y.}$$

> **Why this helps.** The envelope $\mathcal{G}$ is concentrated in a disk of radius $\sim k\,\mathrm{NA}$ centered at $(\tilde{k}_x, \tilde{k}_y) = (0, 0)$ — regardless of α. The PSF patch in (ξ, η) spans only $\sim \lambda/\mathrm{NA}$ in width; its size *does not grow with* α. By contrast, the on-axis Bessel sum converges slowly at the off-axis point $r = f \tan\alpha$.

Polar-to-Cartesian regridding

The FFTLog output gives $\mathcal{U}$ on a polar grid $(k_r^{(n)}, \phi^{(l)})$ with log-spaced k_r and uniform ϕ. The 2D IFFT needs $\mathcal{G}$ on a uniform Cartesian grid centered at $(k \sin\alpha, 0)$. This is accomplished by bilinear interpolation (sufficient when the polar grid is fine) or by a type-2 NUFFT via `NFFT.jl`.

Complete Improved Algorithm

Improved Cy-FFP for Large Oblique Angles (Julia)

Inputs: Near field $\boldsymbol{E}^{\mathrm{Near}}(r,\theta)$ on a log-spaced r-grid and uniform θ-grid; oblique angle α; focal length f; wavenumber $k = 2\pi/\lambda$.

Step 1 — Angular decomposition (`fft` over θ).
For each r_j:

$$E^{\mathrm{N}}_{m,r}(r_j) = \frac{1}{N_\theta}\sum_l E_r(r_j,\theta_l)\, e^{-im\theta_l}, \quad E^{\mathrm{N}}_{m,\theta}(r_j) = \cdots$$

Truncate at $|m| \le M_{\max} = \lceil k \sin\alpha\, R\rceil + \delta M$.

Step 2 — Forward Hankel transforms (`fftlog_hankel`, order $m \pm 1$).
For each $|m| \le M_{\max}$, evaluate Eqs. (1) and (2) using 4 FFTLog calls. Output: $A^{\mathrm{TE}}_m(k_r)$, $A^{\mathrm{TM}}_m(k_r)$ on a log-spaced k_r-grid.

Step 3 — Angular spectrum assembly (`ifft` over m).
At each k_r:

$$\mathcal{U}(k_r,\phi_l) = \frac{2\pi}{k_r}\sum_m (-i)^m \left[A^{\mathrm{TE}}_m(k_r)\,\boldsymbol{e}^{\mathrm{TE}}_m(\phi_l) + A^{\mathrm{TM}}_m(k_r)\,\boldsymbol{e}^{\mathrm{TM}}_m(\phi_l)\right] e^{im\phi_l}.$$

Step 4 — Propagation phase.

$$\mathcal{U}(k_r,\phi) \;\leftarrow\; \mathcal{U}(k_r,\phi)\cdot e^{ik_z f}, \quad k_z = \sqrt{k^2 - k_r^2} \;\; \text{(set to 0 if } k_r > k\text{)}.$$

Step 5 — Polar→Cartesian regridding (`polar_to_cartesian`).
Interpolate $\mathcal{U}(k_r,\phi)$ onto a uniform $(\tilde{k}_x,\tilde{k}_y)$ grid centered at $(k\sin\alpha, 0)$ to form $\mathcal{G}(\tilde{k}_x,\tilde{k}_y)$, multiplied by $e^{i(k\sin\alpha+\tilde{k}_x)x_0}$.

Step 6 — Local PSF via 2D IFFT (`ifft2`).

$$\mathrm{PSF}(\xi,\eta) = e^{ik\sin\alpha\cdot\xi}\cdot\mathcal{F}^{-1}_{2D}[\mathcal{G}](\xi,\eta).$$

Julia Implementation Notes

Package dependencies

```
[deps]
FFTW            = "7a1cc6ca-52ef-59f5-83cd-3a7055c09341"
SpecialFunctions = "276daf66-3868-5448-9aa4-cd146d93841b"
LinearAlgebra   = "37e2e46d-f89d-539d-b4ee-838fcccc9c8e"
Interpolations  = "a98d9a8b-a2ab-59e6-89dd-64a1c18fca59"
```

Listing 1: Project.toml dependencies

FFTLog Hankel transform

The core function implements the discrete Hankel transform via FFTLog for any real order ν:

```
using FFTW, SpecialFunctions
```

```julia

"""
    fftlog_hankel(f_r, dln, nu; kr_min=nothing, bias=0.0)

Compute the Hankel transform of order `nu` of the function `f_r`
sampled at log-spaced points r[j] = r0 * exp(j * dln).

Returns (k_r, A_kr) where k_r is a log-spaced array.
Uses FFTLog: maps to log-space, applies Gamma-function filter, IFFT.
Stable for any real `nu`, including |nu| ~ 600.
"""
function fftlog_hankel(f_r::AbstractVector, dln::Real, nu::Real;
                       bias::Real=0.0)
    N = length(f_r)
    # Fourier modes q = 2*pi*n / (N*dln), n = -N/2 ... N/2-1
    n = fftfreq(N, N)           # FFTW convention: [0,1,...,N/2-1,-N/2,...,-1]
    q = (2pi / (N * dln)) .* n

    # Log-Gamma filter kernel in Fourier space (log-space for stability)
    # U(q) = Gamma((nu+1+q)/2) / Gamma((nu+1-q)/2) * 2^q
    function log_filter(q_val)
        a = (nu + 1 + q_val) / 2
        b = (nu + 1 - q_val) / 2
        return real(loggamma(a) - loggamma(b)) + q_val * log(2)
    end
    log_U = log_filter.(q)
    U = exp.(log_U)             # magnitude of filter
    # Phase comes from complex loggamma: use actual complex ratio
    function filter_complex(q_val)
        a = (nu + 1 + q_val) / 2
        b = (nu + 1 - q_val) / 2
        return exp(loggamma(complex(a)) - loggamma(complex(b)) +
                   q_val * log(2))
    end
    U_c = filter_complex.(q)

    # Apply optional power-law bias: multiply f_r by r^(-bias) in real space
    # (handled via modified exponent in filter)
    # Forward FFT of f_r
    F = fft(f_r)
    # Multiply by filter in Fourier space
    G = F .* U_c
    # Inverse FFT -> A on log-spaced k_r grid (reversed)
    A = real(ifft(G))
    return A
end
```

Listing 2: FFTLog Hankel transform

Angular decomposition (Step 1)

```julia
"""
    angular_decompose(Er, Etheta, M_max)

FFT over theta axis (axis=2) of the near-field arrays Er[Nr, Ntheta]
and Etheta[Nr, Ntheta]. Returns Em_r[Nr, 2M_max+1] and Em_theta[Nr, 2M_max+1]
for modes m = -M_max ... M_max (Julia 1-indexed: index 1 -> m = -M_max).
"""
function angular_decompose(Er::Matrix, Etheta::Matrix, M_max::Int)
    Nr, Ntheta = size(Er)
    Em_r     = fft(Er,     2) ./ Ntheta
    Em_theta = fft(Etheta, 2) ./ Ntheta
    # FFTW ordering: index j corresponds to m = j-1 (for j <= N/2+1)
    # and m = j-1-N (for j > N/2+1). Rearrange to -M_max:M_max.
    modes = [0:M_max; -(M_max:-1:1)]   # FFTW positive/negative freq order
    # Extract only the modes we need
    idx = mod.(modes, Ntheta) .+ 1
    return Em_r[:, idx], Em_theta[:, idx], collect(-M_max:M_max)
end
```

Listing 3: Angular FFT decomposition

Forward Hankel transforms for TE and TM coefficients (Step 2)

```julia
"""
    compute_TE_TM_coeffs(Em_r, Em_theta, m_list, r, k)

For each angular mode m in m_list, compute A_TE_m(kr) and A_TM_m(kr)
using 4 FFTLog calls (Eqs. for TE; 4 more for TM).
`r` is the log-spaced radial grid; `k` = 2*pi/lambda.

Returns A_TE[Nr_out, N_modes], A_TM[Nr_out, N_modes], kr_grid.
"""
function compute_TE_TM_coeffs(Em_r::Matrix{ComplexF64},
                              Em_theta::Matrix{ComplexF64},
                              m_list::Vector{Int},
                              r::Vector{Float64},
                              k::Float64)
    Nr     = length(r)
    dln    = log(r[2] / r[1])
    r_mid  = exp(mean(log.(r)))
    N_modes = length(m_list)

    # Output kr grid has same length as r (FFTLog is symmetric)
    kr_grid = exp.(log(1/r[end]) .+ dln .* (0:Nr-1))

    A_TE = zeros(ComplexF64, Nr, N_modes)
    A_TM = zeros(ComplexF64, Nr, N_modes)

    for (idx, m) in enumerate(m_list)
        # Inputs: r * Em (weight for Hankel transform)
```

```
        f_r  = r .* Em_r[:, idx]       # r * E_r component for mode m
        f_th = r .* Em_theta[:, idx]   # r * E_theta component for mode m

        # --- TE coefficient (Eq. 1) ---
        # A_TE = (i/2)[H_{m+1}(r*E_r) + H_{m-1}(r*E_r)]
        #        + (1/2)[H_{m-1}(r*E_th) - H_{m+1}(r*E_th)]
        Hmp1_r  = fftlog_hankel(f_r,  dln, Float64(m + 1))
        Hmm1_r  = fftlog_hankel(f_r,  dln, Float64(m - 1))
        Hmm1_th = fftlog_hankel(f_th, dln, Float64(m - 1))
        Hmp1_th = fftlog_hankel(f_th, dln, Float64(m + 1))

        A_TE[:, idx] = (im/2) .* (Hmp1_r .+ Hmm1_r) .+
                       (1/2)  .* (Hmm1_th .- Hmp1_th)

        # --- TM coefficient (Eq. 2) ---
        # A_TM = -(kz/k)/2 * [H_{m-1}(r*E_r) - H_{m+1}(r*E_r)]
        #        - i*(kz/k)/2 * [H_{m-1}(r*E_th) + H_{m+1}(r*E_th)]
        # kz/k depends on kr, computed per kr point below
        kz_over_k = sqrt.(max.(1.0 .- (kr_grid ./ k).^2, 0.0))

        A_TM[:, idx] = -(kz_over_k ./ 2) .* (Hmm1_r .- Hmp1_r) .-
                         im .* (kz_over_k ./ 2) .* (Hmm1_th .+ Hmp1_th)
    end
    return A_TE, A_TM, kr_grid
end
```

Listing 4: TE and TM coefficient computation via FFTLog

Angular spectrum assembly and propagation (Steps 3–4)

```
"""
    assemble_angular_spectrum(A_TE, A_TM, m_list, kr_grid, k, f, Nphi)

Assemble U(kr, phi) = (2pi/kr) * sum_m (-i)^m * A_m * e^{im*phi}
via IFFT over m. Then multiply by propagation phase e^{i*kz*f}.
Returns U[Nkr, Nphi] (complex).
"""
function assemble_angular_spectrum(A_TE::Matrix{ComplexF64},
                                   A_TM::Matrix{ComplexF64},
                                   m_list::Vector{Int},
                                   kr_grid::Vector{Float64},
                                   k::Float64, f::Float64,
                                   Nphi::Int)
    Nkr     = length(kr_grid)
    N_modes = length(m_list)
    M_max   = maximum(abs.(m_list))

    # Allocate output
    U = zeros(ComplexF64, Nkr, Nphi)

    # Prepare IFFT buffer of length Nphi (modes mapped to FFTW indexing)
    phi = range(0, 2pi, length=Nphi+1)[1:end-1]

```

```
    for ikr in 1:Nkr
        buf = zeros(ComplexF64, Nphi)
        for (idx, m) in enumerate(m_list)
            phase = (-im)^m
            # TE + TM combined
            coeff = phase * (A_TE[ikr, idx] + A_TM[ikr, idx])
            # Map m to FFTW index
            jj = mod(m, Nphi) + 1
            buf[jj] += coeff
        end
        # IFFT over m -> phi
        u_phi = ifft(buf) .* Nphi .* (2pi / kr_grid[ikr])
        U[ikr, :] = u_phi
    end

    # Propagation phase e^{i*kz*f} (Step 4)
    kz = sqrt.(complex.(k^2 .- kr_grid.^2))
    prop = exp.(im .* kz .* f)
    # Zero out evanescent modes (imaginary kz)
    prop[kr_grid .>= k] .= 0.0
    U .*= prop

    return U, phi
end
```

Listing 5: Angular spectrum assembly and propagation

Polar-to-Cartesian regridding (Step 5)

```
using Interpolations

"""
    polar_to_cartesian(U, kr_grid, phi_grid, k, alpha, x0,
                       Nkx, Nky, dkx, dky)

Interpolate U(kr, phi) from polar onto a uniform Cartesian grid
(ktilde_x, ktilde_y) centered at (k*sin(alpha), 0).
Multiplies by the shift phase e^{i*(k*sin(alpha)+ktx)*x0}.
Returns G[Nkx, Nky] (complex).
"""
function polar_to_cartesian(U::Matrix{ComplexF64},
                            kr_grid::Vector{Float64},
                            phi_grid::Vector{Float64},
                            k::Float64, alpha::Float64,
                            x0::Float64,
                            Nkx::Int, Nky::Int,
                            dkx::Float64, dky::Float64)
    # Build interpolant on (log_kr, phi) grid
    # Use linear interpolation in log(kr) and linear in phi
    log_kr = log.(kr_grid)
    itp = interpolate((log_kr, phi_grid), U,
                      (Gridded(Linear()), Gridded(Linear())))

```

```
    kx_center = k * sin(alpha)
    ktx = range(-(Nkx 2)*dkx, step=dkx, length=Nkx)
    kty = range(-(Nky 2)*dky, step=dky, length=Nky)

    G = zeros(ComplexF64, Nkx, Nky)
    for (ix, kx_tilde) in enumerate(ktx)
        kx = kx_center + kx_tilde
        for (iy, ky_tilde) in enumerate(kty)
            kr  = sqrt(kx^2 + ky_tilde^2)
            phi = atan(ky_tilde, kx)
            if phi < 0; phi += 2pi; end
            # Interpolate U
            if kr <= maximum(kr_grid) && kr >= minimum(kr_grid)
                u_val = itp(log(kr), phi)
            else
                u_val = zero(ComplexF64)
            end
            # Shift phase
            shift = exp(im * kx * x0)
            G[ix, iy] = u_val * shift
        end
    end
    return G, ktx, kty
end
```

Listing 6: Polar to Cartesian regridding centered at the focal spot

Local PSF (Step 6)

```
"""
    compute_local_psf(G, ktx, kty, k, alpha)

Given the shift-centered angular spectrum G[Nkx, Nky] on uniform
(ktx, kty) grid, compute the local PSF via 2D IFFT.
Returns (psf[Nxi, Neta], xi_grid, eta_grid).
The carrier e^{i*k*sin(alpha)*xi} is factored out; the returned
`psf` is the PSF envelope.
"""
function compute_local_psf(G::Matrix{ComplexF64},
                           ktx::AbstractRange, kty::AbstractRange,
                           k::Float64, alpha::Float64)
    dkx = step(ktx)
    dky = step(kty)
    Nkx, Nky = size(G)

    # 2D IFFT (with fftshift to center zero-frequency)
    G_shift = ifftshift(G)
    psf_raw = ifft2(G_shift)     # calls FFTW.ifft(G, [1,2])
    psf_raw = fftshift(psf_raw)

    # Physical coordinate grids (xi = x - x0, eta = y)
    dxi  = 2pi / (Nkx * dkx)
    deta = 2pi / (Nky * dky)
```

```
    xi   = range(-(Nkx 2)*dxi,  step=dxi,  length=Nkx)
    eta  = range(-(Nky 2)*deta, step=deta, length=Nky)

    # Carrier tilt already absorbed into G via the x0 shift phase;
    # the PSF envelope is directly psf_raw * normalization
    norm = dkx * dky / (2pi)^2 * Nkx * Nky
    psf  = psf_raw .* norm

    return psf, xi, eta
end

# Convenience wrapper: 2D IFFT along both dimensions
ifft2(A) = FFTW.ifft(A, [1, 2])
```

Listing 7: Local PSF via 2D IFFT

Top-level driver

```
"""
    cyfft_farfield(Er, Etheta, r, theta, k, alpha, f;
                   M_buffer=10, Nphi=512, Nkx=256, Nky=256)

Full near-to-far-field transform for a vector near field (Er, Etheta)
given on a log-spaced radial grid `r` and uniform angular grid `theta`.

# Arguments
- `Er`, `Etheta` : complex arrays [Nr, Ntheta], near-field components
- `r`            : log-spaced radial grid [m]
- `theta`        : uniform angular grid [rad], length Ntheta
- `k`            : wavenumber 2*pi/lambda [1/m]
- `alpha`        : oblique angle [rad]
- `f`            : focal length [m]
- `M_buffer`     : extra angular modes beyond M_max (default 10)
- `Nphi`         : angular resolution in k-space (default 512)
- `Nkx`, `Nky`  : Cartesian k-grid size for PSF (default 256 x 256)

# Returns
- `psf`          : complex PSF envelope in local coordinates [Nkx, Nky]
- `xi`, `eta`    : local coordinate grids [m]
"""
function cyfft_farfield(Er::Matrix{ComplexF64},
                        Etheta::Matrix{ComplexF64},
                        r::Vector{Float64},
                        theta::Vector{Float64},
                        k::Float64, alpha::Float64, f::Float64;
                        M_buffer::Int=10,
                        Nphi::Int=512,
                        Nkx::Int=256, Nky::Int=256)

    R     = maximum(r)
    M_max = ceil(Int, k * sin(alpha) * R) + M_buffer

    # Step 1: angular decomposition
```

```
    Em_r, Em_theta, m_list = angular_decompose(Er, Etheta, M_max)

    # Step 2: forward Hankel transforms (FFTLog)
    A_TE, A_TM, kr_grid = compute_TE_TM_coeffs(
        Em_r, Em_theta, m_list, r, k)

    # Step 3 & 4: angular spectrum assembly + propagation
    x0  = f * tan(alpha)
    U, phi_grid = assemble_angular_spectrum(
        A_TE, A_TM, m_list, kr_grid, k, f, Nphi)

    # Step 5: polar -> Cartesian centered at (k*sin(alpha), 0)
    dkx = k * sin(alpha) / (Nkx / 2)   # Nyquist-based spacing
    dky = dkx
    G, ktx, kty = polar_to_cartesian(
        U, kr_grid, phi_grid, k, alpha, x0, Nkx, Nky, dkx, dky)

    # Step 6: local PSF via 2D IFFT
    psf, xi, eta = compute_local_psf(G, ktx, kty, k, alpha)

    return psf, xi, eta
end
```

Listing 8: Top-level: full far-field propagation

Comparison Table

Quantity	Original	Improved (this work)
Modes required	$M_{\max} \sim k \sin \alpha R$	Same (physics-determined)
HT algorithm	Direct / NUFHT (fails $\nu \gtrsim 10$)	FFTLog via `fftlog_hankel` (any ν)
HT cost per mode	$O(N^2)$ or fragile	$O(N \log N)$
Bessel turning point	Problem at $k_r r \approx m$	Not a problem
PSF output domain	Full xy-plane	Local patch, size $\sim (\lambda/\mathrm{NA})^2$
PSF computation	Bessel sum at large r	2D IFFT
Grid size vs α	Grows with α	Fixed
Julia packages	—	`FFTW`, `SpecialFunctions`, `Interpolations`

Symmetry reduction

For $\phi = 0$ illumination, the original notes prove $\boldsymbol{E}_{-m} = \hat{\sigma} \boldsymbol{E}_m$, halving the mode count. This applies unchanged: compute $A_m^{\mathrm{TE/TM}}$ only for $m \geq 0$, then reconstruct $A_{-m}^{\mathrm{TE/TM}}$ via the symmetry.

Large-angle validity

The formulation is exact (no paraxial approximation) provided: (1) the near-field plane lies in the propagating region; (2) $M_{\max}$ is sufficient; (3) the evanescent cutoff is applied in Step 4.